Caso Práctico: Neurona de McCulloch y Pitts



Daniela Maldonado

Git hub: https://github.com/danimaldonado18/machinelearning2

Aplicando la MPNeuronaa un caso práctico real

Conjunto de datos

Esta es una copia de los conjuntos de datos de UCI ML Breast Cancer Winconsin https://archive.ics.uci.edu/dataset/17/breast+cancer+wisconsin+diagnost

Descripción de la muestra

Las caracteristicas de netrada se calculan a partir de una imagen digitaizada de un aspirado de aguja fina (FNA) de una masa mamaria. Describen las caracteristicas de los npucles celulares presentes en la imagén. El plano de separación descrito anteriormente se obtuvo utilizando el m'petodo de árbol de multiples superficies (MSM-T) "COnstrucción de un árbol de decisión mediante programación lineal). El programa lineal utilizado para obtener el plano de separación en el espacio tridimensional es el que describe en [K.P Bennet y O. L Mangasarian]

Lectura de datos

In [2]:
```
from sklearn.datasets import load_breast_cancer
cancer_mama= load_breast_cancer()
# Separamos Los datos de entra y los datos de salida
X= cancer_mama.data
Y=cancer_mama.target
```

In [3]:
```
dir(cancer_mama)
```

Out[3]:
```
['DESCR',
 'data',
 'data_module',
 'feature_names',
 'filename',
 'frame',
 'target',
 'target_names']
```

In [4]:
```
print(X)
```

```
[[1.799e+01 1.038e+01 1.228e+02 ... 2.654e-01 4.601e-01 1.189e-01]
 [2.057e+01 1.777e+01 1.329e+02 ... 1.860e-01 2.750e-01 8.902e-02]
 [1.969e+01 2.125e+01 1.300e+02 ... 2.430e-01 3.613e-01 8.758e-02]
 ...
 [1.660e+01 2.808e+01 1.083e+02 ... 1.418e-01 2.218e-01 7.820e-02]
 [2.060e+01 2.933e+01 1.401e+02 ... 2.650e-01 4.087e-01 1.240e-01]
 [7.760e+00 2.454e+01 4.792e+01 ... 0.000e+00 2.871e-01 7.039e-02]]
```

Visualización del conjunto de datos

In [5]:
```
import pandas as pd
df=pd.DataFrame(X, columns=cancer_mama.feature_names)
df
```

Out[5]:

	mean radius	mean texture	mean perimeter	mean area	mean smoothness	mean compactness	mean concavity	cor p
0	17.99	10.38	122.80	1001.0	0.11840	0.27760	0.30010	0.
1	20.57	17.77	132.90	1326.0	0.08474	0.07864	0.08690	0.
2	19.69	21.25	130.00	1203.0	0.10960	0.15990	0.19740	0.
3	11.42	20.38	77.58	386.1	0.14250	0.28390	0.24140	0.
4	20.29	14.34	135.10	1297.0	0.10030	0.13280	0.19800	0.
...	...	...	...	...	...	...	...	
564	21.56	22.39	142.00	1479.0	0.11100	0.11590	0.24390	0.
565	20.13	28.25	131.20	1261.0	0.09780	0.10340	0.14400	0.
566	16.60	28.08	108.30	858.1	0.08455	0.10230	0.09251	0.
567	20.60	29.33	140.10	1265.0	0.11780	0.27700	0.35140	0.
568	7.76	24.54	47.92	181.0	0.05263	0.04362	0.00000	0.

569 rows × 30 columns

In [6]:
```
print(Y)
```

```
[0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 1 0 0 0 0 0 0 0 0 1 0 1 1 1 1 1 0 0 1 0 0 1 1 1 1 0 1 0 0 1 1 1 1 0 1 0 0
 1 0 1 0 0 1 1 1 0 0 1 0 0 0 1 1 1 0 1 1 0 0 1 1 1 0 0 1 1 1 1 0 1 1 0 1 1
 1 1 1 1 1 1 0 0 0 1 0 0 1 1 1 0 0 1 0 1 0 0 1 0 0 1 1 0 1 1 0 1 1 1 1 0 1
 1 1 1 1 1 1 1 1 0 1 1 1 1 0 0 1 0 1 1 0 0 1 1 0 0 1 1 1 1 0 1 1 0 0 0 1 0
 1 0 1 1 1 0 1 1 0 0 1 0 0 0 0 1 0 0 0 1 0 1 0 1 1 0 1 0 0 0 0 1 1 0 0 1 1
 1 0 1 1 1 1 1 1 0 0 1 1 0 1 1 0 0 1 0 1 1 1 1 0 1 1 1 1 1 0 1 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 1 1 1 1 1 1 1 0 1 0 1 1 0 1 1 0 1 0 0 1 1 1 1 1 1 1 1 1 1 1
 1 0 1 1 0 1 0 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 0 1 1 1 0 1 0 1 1 1 1 0 0 0 1 1
 1 1 0 1 0 1 0 1 1 1 0 1 1 1 1 1 1 1 0 0 0 1 1 1 1 1 1 1 1 1 1 0 0 1 0 0
 0 1 0 0 1 1 1 1 1 0 1 1 1 1 1 0 1 1 1 0 1 1 0 0 1 1 1 1 1 1 0 1 1 1 1 1 1
 1 0 1 1 1 1 1 0 1 1 0 1 1 1 1 1 1 1 1 1 1 1 1 0 1 0 0 1 0 1 1 1 1 1 0 1 1
 0 1 0 1 1 0 1 0 1 1 1 1 1 1 1 1 0 0 1 1 1 1 1 1 0 1 1 1 1 1 1 1 1 1 1 0 1
 1 1 1 1 1 1 0 1 0 1 1 0 1 1 1 1 1 1 0 0 1 0 1 0 1 1 1 1 1 0 1 1 0 1 0 1 0 0
 1 1 1 0 1 1 1 1 1 1 1 1 1 1 1 0 1 0 0 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
 1 1 1 1 1 1 1 0 0 0 0 0 0 1]
```

División del conjunto de datos

In [7]:
```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(df, Y, stratify= Y)
print("Tamaño de entrenamiento ", len(X_train))
print("Tamaño de pruebas ", len(X_test))
```

```
Tamaño de entrenamiento  426
Tamaño de pruebas  143
```

Implementación de un aMP Neurona más avanzada

In [11]:
```
# importamos la biblioteca de numpy
import numpy as np
from sklearn.metrics import accuracy_score

class MpNeurona:
    # creamos nuestro constructor
    def _init_(self):
        self.threshold=None

    # Creamos una función par el modelo
    def model(self, x):
        #input: [1,0,1,0][x1,x2,...xn]
        z=sum(x)
        return (z >= self.threshold)

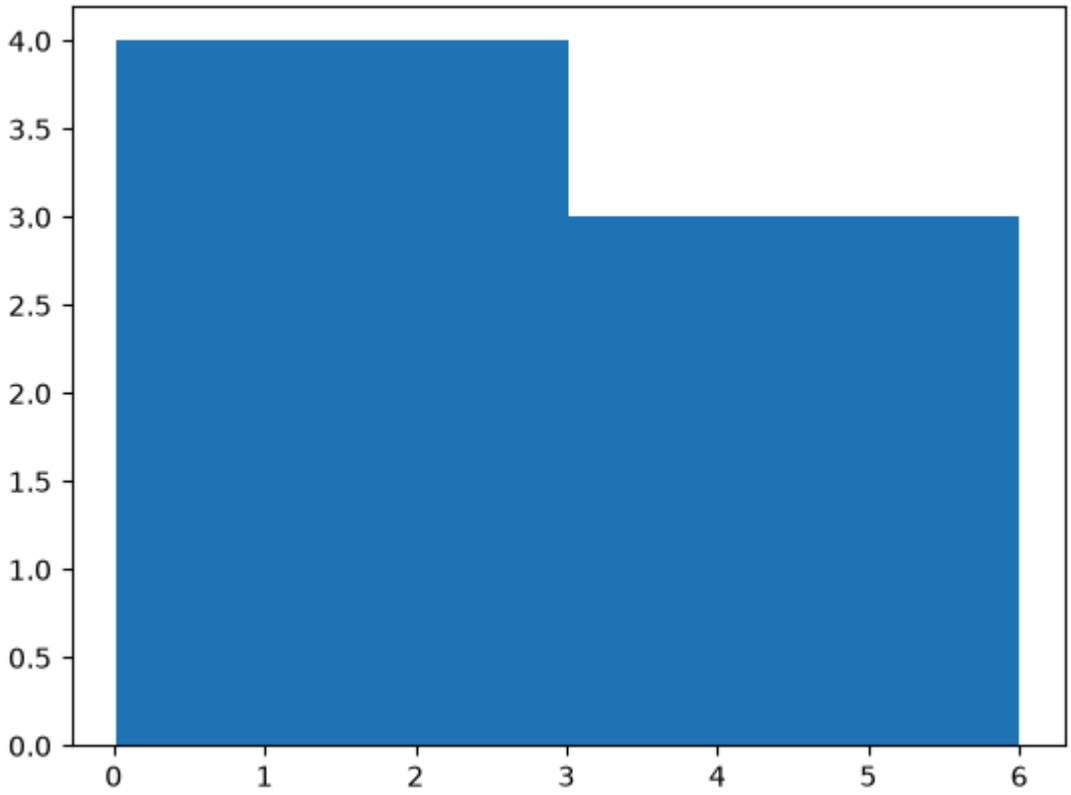
    #Creamos la predicción
    def predict(self, X):
        # input:[[1,0,1,0][1,0,1,1]]
        Y=[]
        for x in X:
            result=self.model(x)
            Y.append(result)
        return np.array(Y)
    # función de entrenamiento para e lthreshold
    def fit(self, X, Y):
        accuracy={}
        #Seleccionamos un threshold entre el # de caracteristicas de entr
        for th in range(X.shape[1]+1):
            self.threshold = th
            Y_pred = self.predict(X)
            accuracy[th]=accuracy_score(Y_pred, Y)
    #Seleccionamos el threshold que mejores resultado proporciona
        self.threshold = max(accuracy, key=accuracy.get)
```

Los datos de entrada siguen teniendo porblemas debido a que nuestro conjunto de datos las caracteristicas de entreda reciben valores continuos, sin embargo, la neurona MP solo procesa características de netrada con valores binarios

In [12]:
```
#Para transformar un valor a binario
import matplotlib.pyplot as plt
print(pd.cut([0.04,0.3,4,5,6,0.02,0.6], bins=2, labels=[0,1]))

plt.hist([0.04,0.3,4,5,6,0.02,0.6], bins=2)
plt.show()
```

```
[0, 0, 1, 1, 1, 0, 0]
Categories (2, int64): [0 < 1]
```



In [13]:
```
#Trasformarmamos las caracteristicas de entrada a un valor binario
X_train_bin= X_train.apply(pd.cut, bins=2, labels=[1,0])
X_test_bin = X_test.apply(pd.cut, bins=2, labels=[1,0])

X_train_bin
```

Out[13]:

	mean radius	mean texture	mean perimeter	mean area	mean smoothness	mean compactness	mean concavity	m con po
89	1	1	1	1	0	1	1	
41	1	1	1	1	0	1	1	
138	1	1	1	1	0	1	1	
342	1	1	1	1	0	1	1	
487	0	1	0	1	0	1	0	
...	...	...	...	...	...	...	...	
491	0	1	1	1	1	1	1	
362	1	1	1	1	1	1	1	
86	1	1	1	1	1	1	1	
247	1	1	1	1	1	1	1	
357	1	1	1	1	1	1	1	

426 rows × 30 columns

In [14]:
```
# instaciamos el modelo MP
np_neurona = MpNeurona()
#Encontramos el threshold optimo
```

```
np_neurona.fit(X_train_bin.to_numpy(), y_train)
# threshold optimo seleccionado
np_neurona.threshold
```

Out[14]: 27

In [15]:
```
# Realizamos la predicción par alos datos de prueba
Y_pred=np_neurona.predict(X_test_bin.to_numpy())
```

In [16]:
```
Y_pred
```

Out[16]:
```
array([ True, False, False, False,  True, False,  True,  True,  True,
       False,  True,  True,  True,  True,  True, False,  True,  True,
        True, False, False,  True,  True, False,  True,  True,  True,
       False, False, False,  True,  True,  True, False,  True, False,
       False,  True, False,  True, False,  True,  True,  True, False,
       False,  True,  True,  True, False,  True, False, False,  True,
        True,  True, False,  True,  True, False,  True, False,  True,
       False, False, False,  True,  True,  True, False,  True,  True,
       False,  True, False,  True,  True,  True,  True,  True, False,
        True,  True,  True, False,  True, False, False,  True,  True,
       False,  True,  True, False,  True, False, False,  True, False,
       False,  True, False,  True,  True,  True,  True, False,  True,
        True, False, False, False, False, False,  True,  True,  True,
        True,  True,  True,  True, False,  True,  True,  True,  True,
        True, False, False,  True, False, False, False,  True, False,
        True, False, False,  True, False,  True,  True,  True])
```